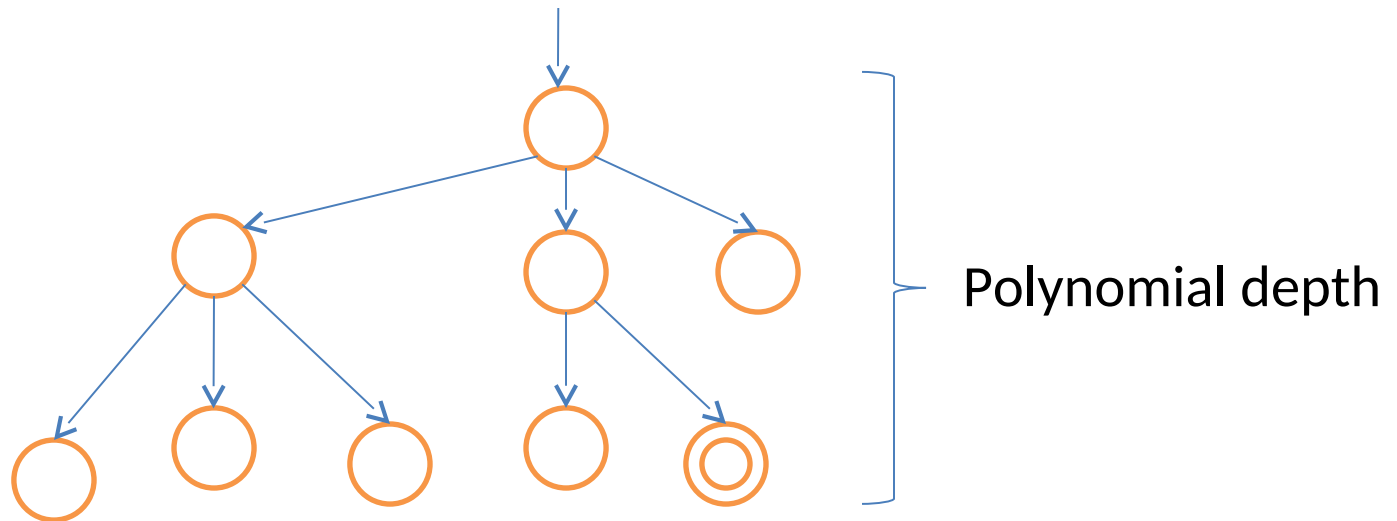


The NP class. NP-completeness

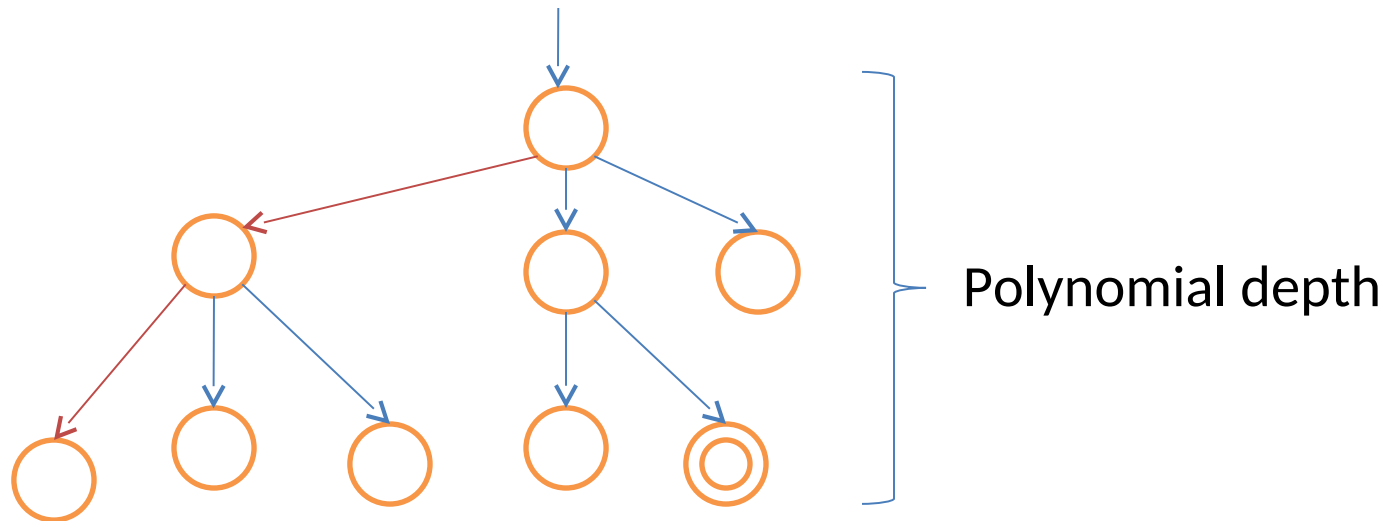
The NP-class

- The NP class is a class that contains all the problems that can be decided by a Non-Deterministic Turing Machine in polynomial time. In other words, the longest accepting path of the machine is going to be polynomial in terms of the size of the input.



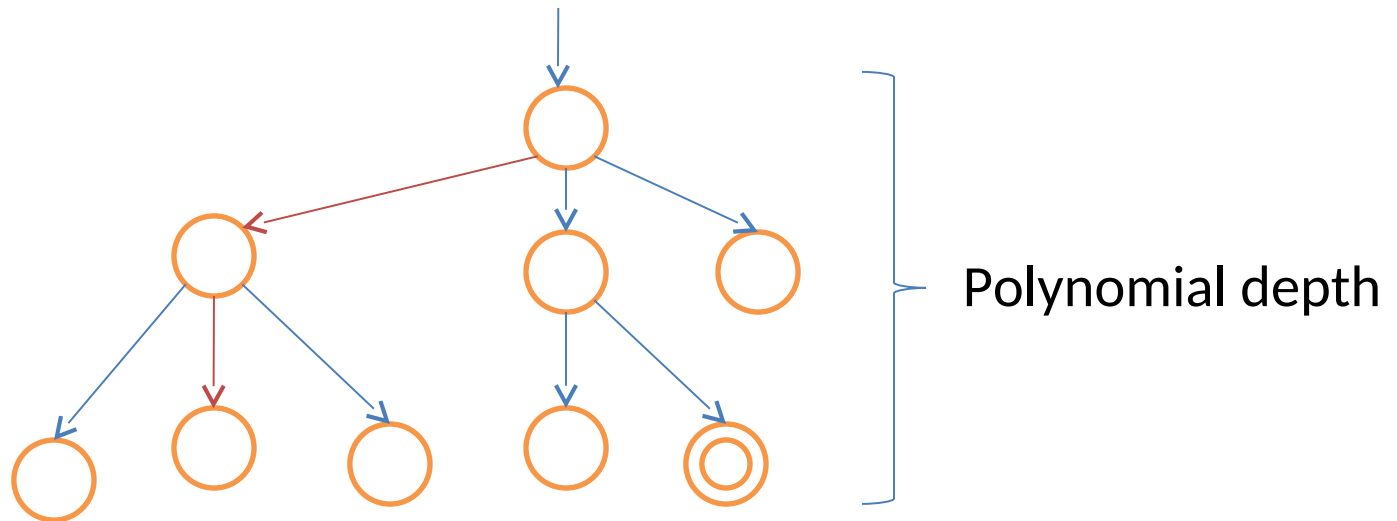
The NP-class

- As we know, a non-deterministic machine is not a real machine since it performs choices. In order to simulate such a machine we have to check every possible path one-by-one to see if any of them is an accepting one. This might take exponential time.



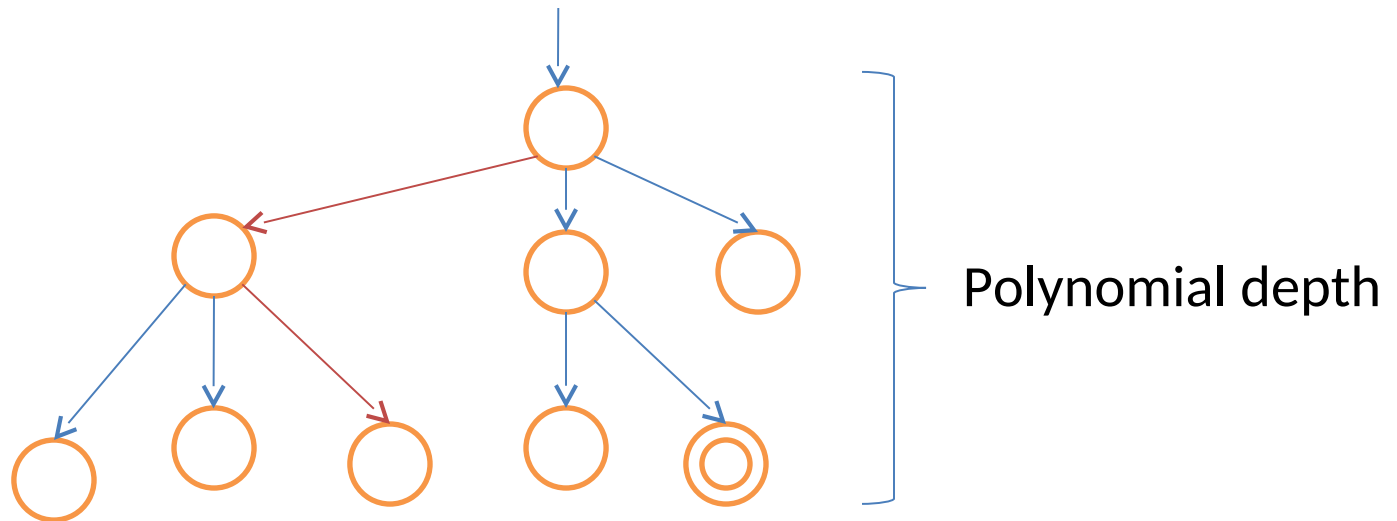
The NP-class

- As we know, a non-deterministic machine is not a real machine since it performs choices. In order to simulate such a machine we have to check every possible path one-by-one to see if any of them is an accepting one. This might take exponential time.



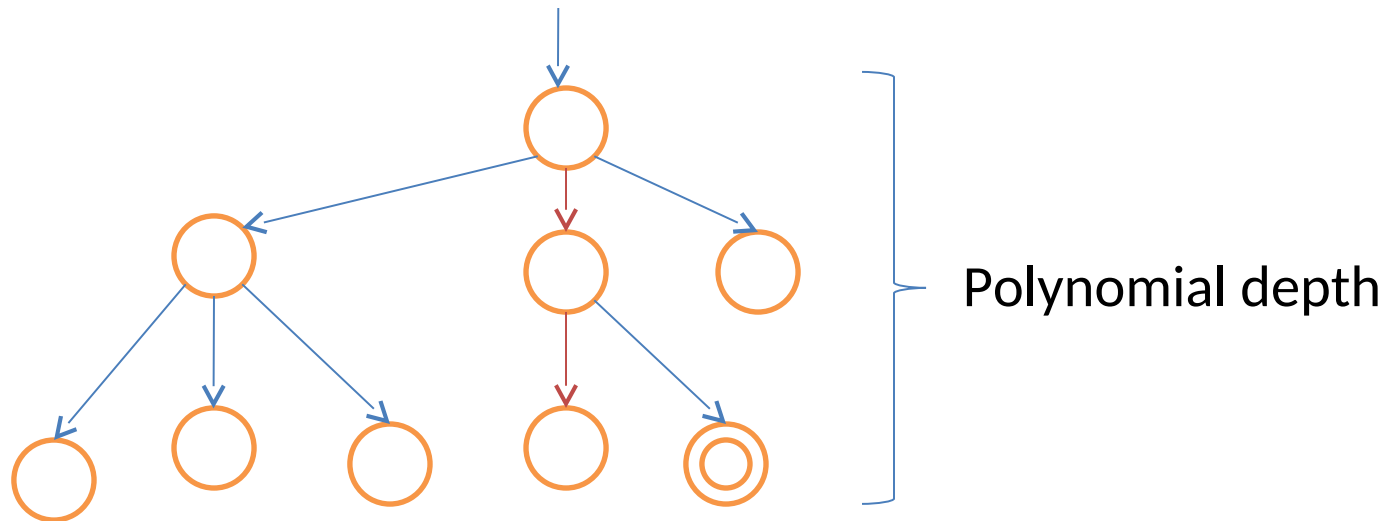
The NP-class

- As we know, a non-deterministic machine is not a real machine since it performs choices. In order to simulate such a machine we have to check every possible path one-by-one to see if any of them is an accepting one. This might take exponential time.



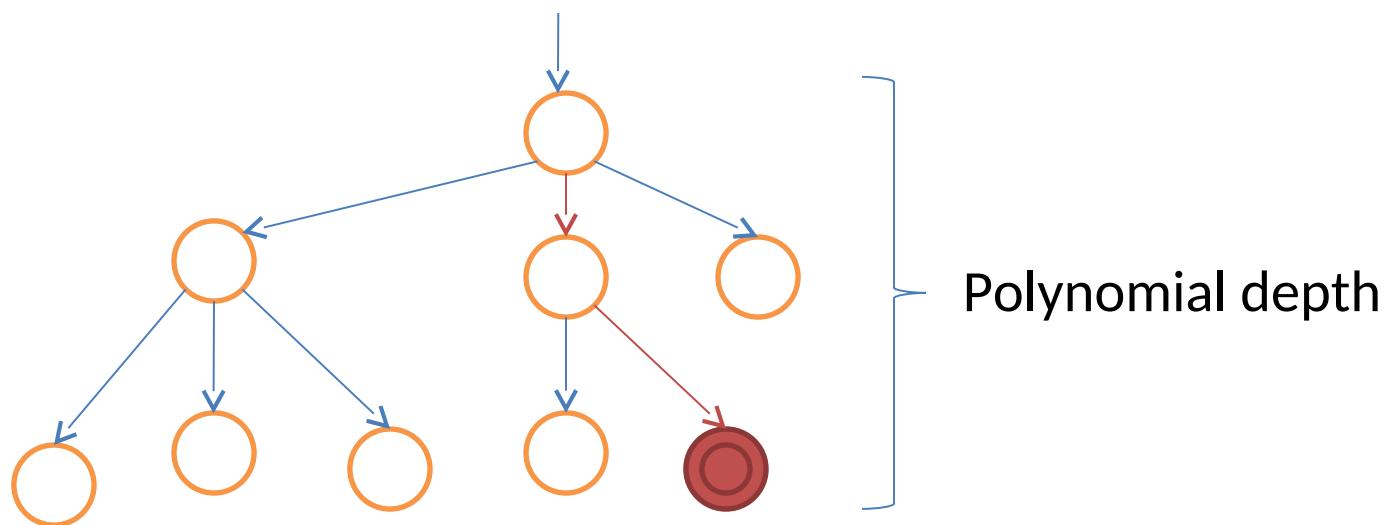
The NP-class

- As we know, a non-deterministic machine is not a real machine since it performs choices. In order to simulate such a machine we have to check every possible path one-by-one to see if any of them is an accepting one. This might take exponential time.



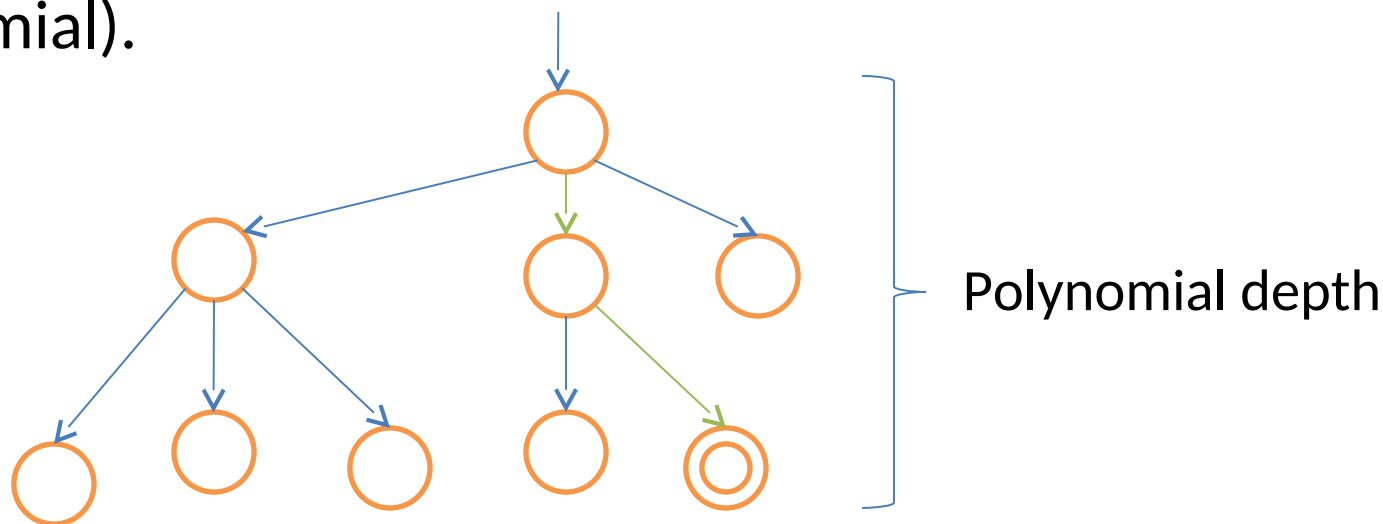
The NP-class

- As we know, a non-deterministic machine is not a real machine since it performs choices. In order to simulate such a machine we have to check every possible path one-by-one to see if any of them is an accepting one. This might take exponential time.



The NP-class

- If there is no accept path under a specific input we should indeed check every path in order to be sure that the answer is “reject” (there is no other way to check it).
- However if the answer is “accept”, there is a polynomial time way in order to verify it: If somebody gives us the accept path (or we guess the correct choices) we can follow the path in polynomial time (since its length is polynomial).



The NP-class

- Thus, an equivalent characterization of the NP class is that it is the class of problems that have a polynomial time verifier: If the instance is a “yes” instance and somebody gives us the correct choices we can verify in polynomial time that it is indeed a “yes” instance.
- You can think of this idea as a student who wants to solve a difficult problem. He needs plenty of time in order to solve it, trying every possible method that he knows. However, if the teacher solves it on the blackboard, he can follow the quick proof (hopefully) without any problems (he can verify quickly).

The NP-class

- Some NP problems:
 - SAT: Given a formula in CNF is it satisfiable?
 - Vertex Cover: Given a graph G and a number k , is there a vertex cover of size k in G ?
 - Independent Set: Given a graph G and a number k , is there an independent set of size k in G ?
 - Clique: Given a graph G and a number k , is there a clique of size k in G ?
 - Hamilton Path: Given a graph G does it have a Hamilton path?

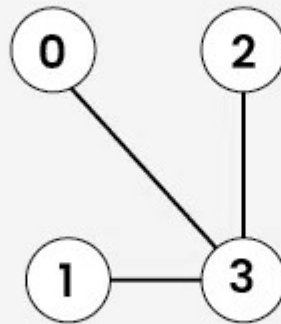
Vertex Cover

A vertex cover of an undirected graph is a subset of its vertices such that for every edge (u, v) of the graph, either 'u' or 'v' is in the vertex cover. Although the name is Vertex Cover, the set covers all edges of the given graph. ***Given an undirected graph, the vertex cover problem is to find minimum size vertex cover.***

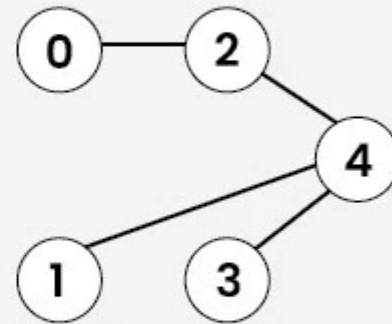
Vertex Cover Problem



Minimum vertex cover is $\{\}$



Minimum vertex cover is $\{3\}$

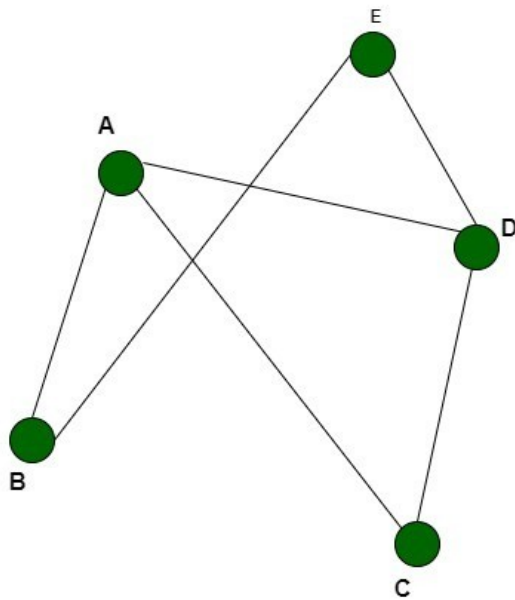


Minimum vertex cover is $\{4, 2\}$ or $\{4, 0\}$

Independent Set

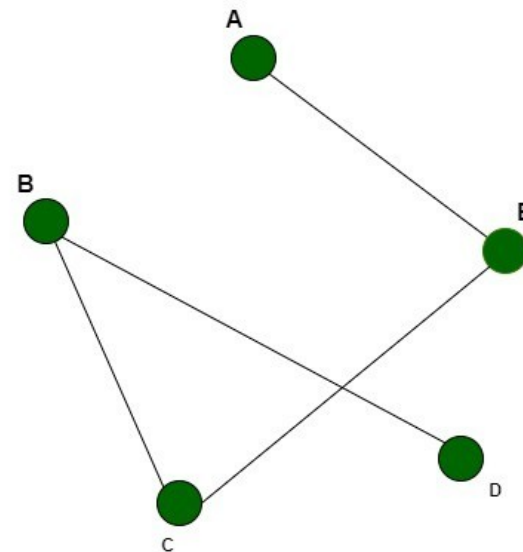
A set of vertices I is called an independent set if no two vertices in set I are adjacent to each other in other words the set of non-adjacent vertices is called an independent set.

Problem: Given a graph $G(V, E)$ and an integer k , the problem is to determine if the graph contains an independent set of vertices of size $\geq k$. **Explanation:** An instance of the problem is an input specified to the problem. An instance of the Independent Set problem is a graph $G(V, E)$ and a positive integer k , and the problem is to check whether an independent set of size k exists in G



$G \cong$

Clique of size 3 {A C D}

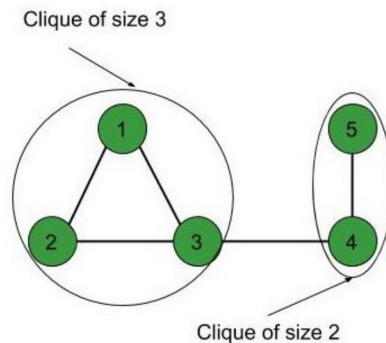


$G' \cong$

Independent Set of size 3 {A C D}

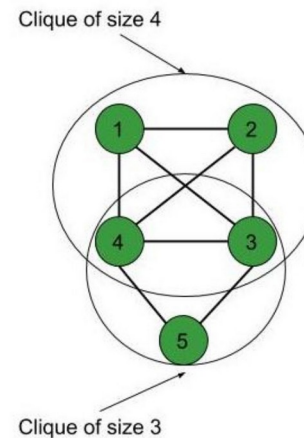
Clique

- A clique is a subgraph of a graph such that all the vertices in this subgraph are connected with each other that is the subgraph is a complete graph. The Maximal Clique Problem is to find the maximum sized clique of a given graph G , that is a complete graph which is a subgraph of G and contains the maximum number of vertices. This is an optimization problem. Correspondingly, the Clique Decision Problem is to find if a clique of size k exists in the given graph or not.



The above graph contains a maximum clique of size 3

Fig. (1)



The above graph contains a maximum clique of size 4

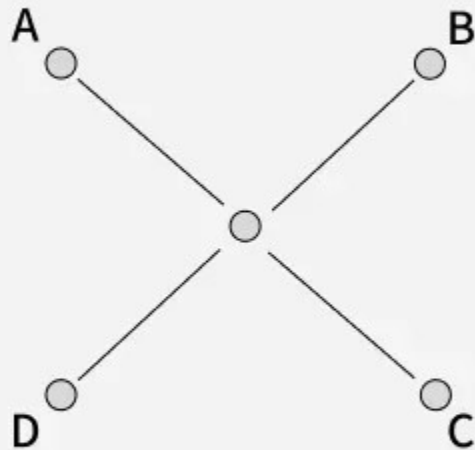
Fig. (2)

* A clique of size 2 is also present in Fig. (2)

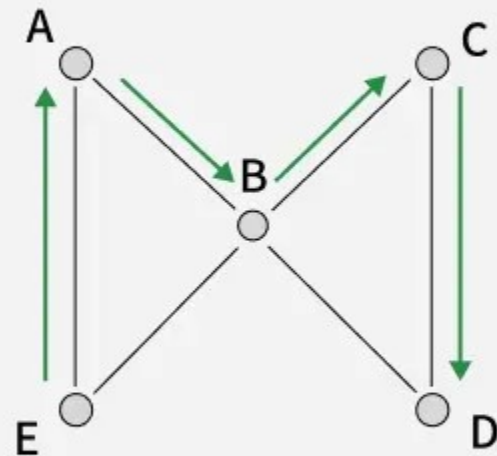
Hamiltonian Path

- ***Hamiltonian Path:*** A path in a graph that visits every vertex exactly once.

Hamiltonian Path: A path in a graph that visits every vertex exactly once.



Hamiltonian Path
Not Possible

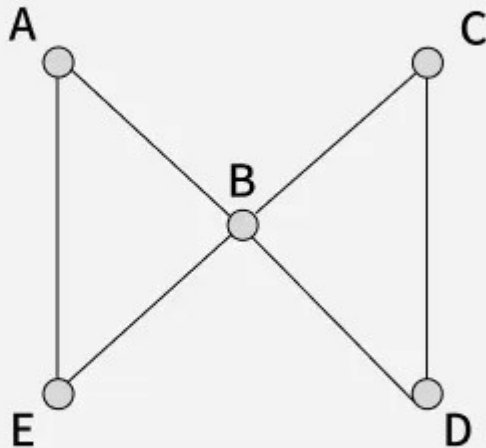


Hamiltonian path
E-A-B-C-D

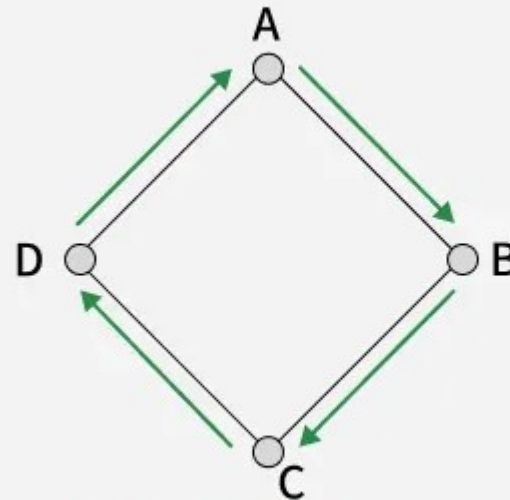
Hamiltonian Circuit

- A simple circuit in a graph G that passes through every vertex exactly once is called a Hamiltonian circuit.*

Hamiltonian Circuit: A circuit that passes through every vertex exactly once.



Hamiltonian Circuit
Not Possible



Hamiltonian Circuit
A-B-C-D-A

Subset Sum

- Given **N** non-negative integers **$a_1 \dots a_N$** and a target sum **K**, the task is to decide if there is a subset having a sum equal to **K**.

Input: *arr[] = [3, 34, 4, 12, 5, 2], sum = 9*

Output: *True*

Explanation: *There is a subset (4, 5) with sum 9.*

- Input:** *arr[] = [3, 34, 4, 12, 5, 2], sum = 30*

Output: *False*

Explanation: *There is no subset that add up to 30.*

The class NP

- Generally, the NP problems are those problems that despite the fact that we can easily check with some help if the answer is yes we are going to need much time in order to solve them.
- We don't know if there exists a polynomial time algorithm for many problems in NP. Actually, this is one of the most interesting open questions in the field of theoretical computer science.

The NP class

- We know that P is a subset of NP since P is the class of problems that can be decided by a Deterministic Turing Machine in Polynomial time and a DTM is by definition an NTM.
- What we don't know is if $P=NP$ (in other words if all the problems in NP can be decided in polynomial time by a DTM).

SAT

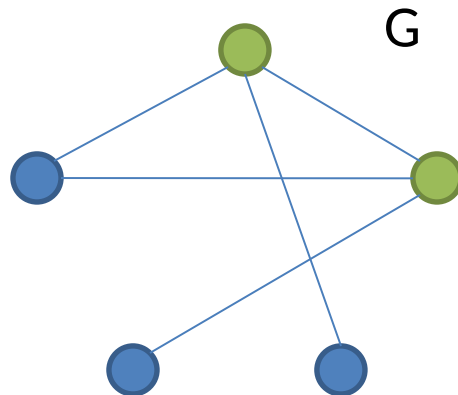
- Given a formula in CNF, is it satisfiable?
- A formula in CNF contains clauses that are connected with “and”. Each clause contains variables that are connected with “or”.
- A formula is satisfiable if there is an assignment on the variables such that the formula is satisfied (gets the value TRUE)

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

Setting x_1 to False, x_2 to False and x_3 to True satisfies the formula

Vertex Cover

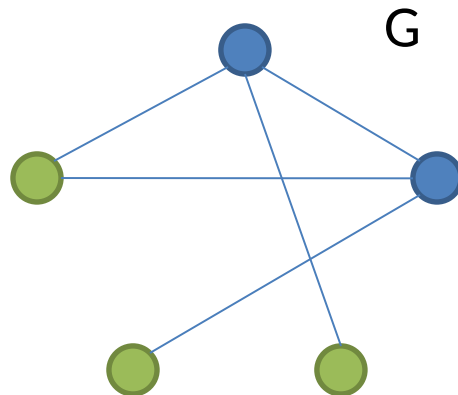
- Given a graph G and a number k does the graph have a vertex cover of size k ?
- A vertex cover is a set of nodes such that any edge of the graph has at least one end point belonging in the set.



The green set of nodes is a vertex cover of G .

Independent Set

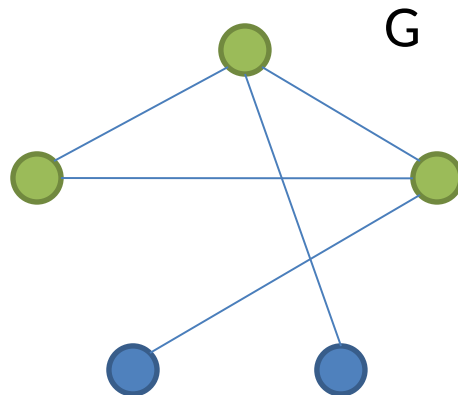
- Given a graph G and a number k does the graph have an independent set of size k ?
- An independent set is a set of nodes such that all pairs of nodes are not connected.



The green set of nodes is an independent set of G .

Clique

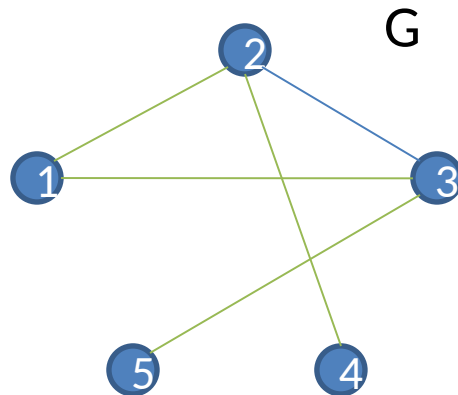
- Given a graph G and a number k does the graph have a clique of size k ?
- A clique is a set of nodes such that any pair of nodes in the set is connected.



The green set of nodes is a clique of G .

Hamilton Path

- Given a graph G does the graph have a Hamilton path?
- A Hamilton path is a permutation of the nodes such that consecutive nodes are connected (the permutation forms a path).



4,2,1,3,5 is a Hamilton path of G .

The Independent Set is in NP

- We can verify in polynomial time a “yes” instance (a graph G that has an independent set of size k). If we are given a set of nodes S of size k (or we guess it) we can check that it is indeed an independent set by checking in the adjacency matrix of G if any two elements i, j in S are disconnected ($A[i, j] = 0$). This needs time $O(k^2)$ which is $O(n^2)$ since k can be at most n .

Optimization problems

- So far we were talking about problems with yes-no answers.
- Optimization problems are also of interest!
- Optimization problems:
 - Minimization (minimize an objective function);
 - Maximization (maximize an objective function).

Optimization problems- Example

- OPT-VC: Given a graph G find the minimum k such that there is a vertex cover with k vertices.
- OPT-Clique: Given a graph G find the maximum k such that there is a k -clique.
- OPT-IS: Given a graph G find the maximum k such that there is an independent set of size k .

NP Optimization problems

- Observe that for an optimization problem that is in NP, k should be at most exponential on the size of the input (we should be able to express k in binary in polynomial time otherwise we won't be able to produce it).
- NP Optimization problems have the same difficulty as their yes-no version (meaning that the optimization version is reducible to the yes-no and vice versa).

Yes-no problems to minimization problems

- *If we have a polynomial time algorithm for a minimization problem in NP then we can obtain a polynomial time algorithm for the yes-no version of the problem.*

Find the optimal solution and if it is larger (worse) than the bound say no, else reply yes.

Minimization problems to yes-no problems

- *If we have a polynomial time algorithm for a yes-no problem in NP then we can obtain a polynomial time algorithm for the optimization version of the problem.*

Idea: Try all values $k=1, 2, 3, \dots$ and the first that replies yes is the minimum.

Minimization problems to yes-no problems

- *If we have a polynomial time algorithm for a yes-no problem in NP then we can obtain a polynomial time algorithm for the optimization version of the problem.*

This might take exponential time in the size of the input since we run the problem k times (recall that k can be exponential in the size of the input).

Minimization problems to yes-no problems

- *If we have a polynomial time algorithm for a yes-no problem in NP then we can obtain a polynomial time algorithm for the optimization version of the problem.*

Instead of trying all possible values we do a trick -binary search- that reduces the number of repetitions in $\log k$ (which is as we said at most polynomial in the size of the input).

Thus we run the polynomially solvable yes-no problem polynomially many times.

Reducibility revisited

- A decision problem A is called polynomially Karp-reducible to a decision problem B (we write $A \leq B$) if there is a polynomial time function $f: A \rightarrow B$ such that
 - if x is a “yes” instance of A then $f(x)$ is a “yes” instance of B and
 - if x is a “no” instance of A then $f(x)$ is a “no” instance of B .
- In simple words, this means that there is an efficient way to transform any instance of A to an instance of B with the same answer.

Reducibility

- Knowing that $A \leq_p B$ could be useful for two reasons:
 - If we have a polynomial time algorithm for solving B then we can solve A in polynomial time: we transform any instance of A to an instance of B using f (polynomial), solve B (polynomial) and then reply what the algorithm for B outputs.
 - If we know for some reason that A cannot be solved in polynomial time we can conclude that B cannot be solved in polynomial time, because what the above case says is that if we could solve B in polynomial time then A could be solved in polynomial time too.

Reductions

- Independent Set $\leq_p$ Clique

Suppose that we have an instance of IS (a graph G and a number k). We create an instance of Clique as follows:

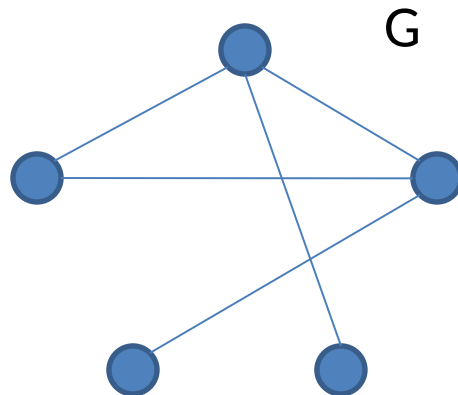
We take as graph the complement of G (G^c) and as clique number again k .

Observe that this transformation can be done in polynomial time (to take the complement of G is the same as exchanging 0 with 1 in the adjacency matrix, except from the diagonal, so the time needed is $O(n^2)$).

Reductions

- Independent Set $\leq_p$ Clique

Furthermore, observe that if (G, k) is a “yes” instance of the IS (there is an independent set of size k in G) then (G^c, k) is also a “yes” instance of Clique and vice versa.



Reductions

- Independent Set $\leq_p$ Clique

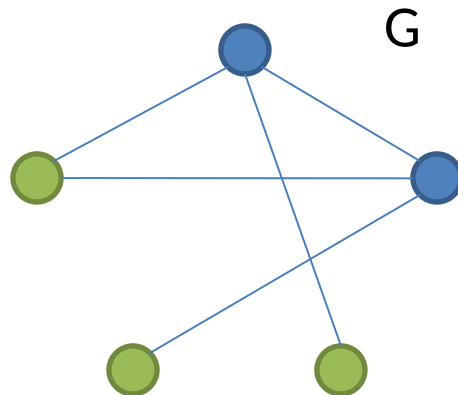
Furthermore, observe that if (G, k) is a “yes” instance of the IS (there is an independent set of size k in G) then (G^c, k) is also a “yes” instance of Clique and vice versa.

- An independent set in G is a set of nodes that have no edges connecting them. All the edges that are missing in G are there in G^c so exactly the same set of nodes is going to be a clique in G^c .
- If there is no independent set of size k in G that means that for all possible choices of k nodes in G there is going to be at least one edge connecting two nodes. This edge is going to be missing in G^c so there is no clique of size k in G^c .

Reductions

- Independent Set $\leq_p$ Clique

Furthermore, observe that if (G, k) is a “yes” instance of the IS (there is an independent set of size k in G) then (G^c, k) is also a “yes” instance of Clique and vice versa.

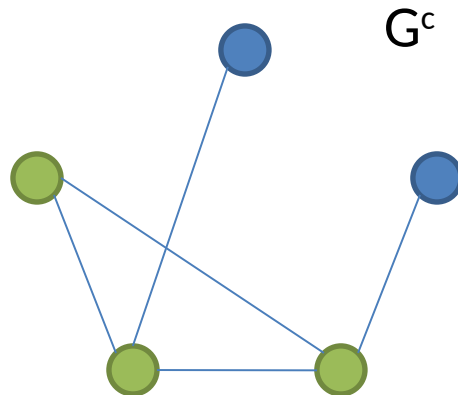


There is an independent set of size 3 in G but there is no independent set of size 4.

Reductions

- Independent Set $\leq_p$ Clique

Furthermore, observe that if (G, k) is a “yes” instance of the IS (there is an independent set of size k in G) then (G^c, k) is also a “yes” instance of Clique and vice versa.



In G^c there is a clique of size 3 but no clique of size 4.

NP-hardness

- We call NP-hard any problem A that all the NP problems are polynomially reducible to A .
- For all B in NP, $B \leq_p A$.
- In other words a problem is called NP-hard if it is at least as hard to solve as any other problem inside the class NP (if we could solve A in polynomial time any NP problem could be solved in polynomial time by reducing it to A).

NP-completeness

- A problem C is NP-complete if:
 - C is in NP
 - C is NP-hard.

The NP-complete problems are the most difficult problems in the class NP by the sense that if C is NP-complete and a polynomial time algorithm is found for it, we can solve any other NP problem by reducing it to C and then solving C .

NP-completeness

- We can show that an NP problem C is NP-complete by reducing an already know NP-complete problem B to it:
 - Since B is NP-complete it holds that for all A in NP, $A \leq_p B$. So for every NP problem A there is a polynomial time function $f_A : A \rightarrow B$ such that we can transform any instance of A to an instance of B with the same answer.
 - If we show that $B \leq_p C$ then there is a polynomial time function $g: B \rightarrow C$ such that we can transform any instance of B to an instance of C with the same answer

NP-completeness

- That means that for any NP problem A there is a polynomial time transformation of any instance of A to an instance of C with the same answer: Use f_A in order to create an instance of B with the same answer and then use g to create an instance of C with the same answer.
- So for all A in NP, $A \leq_p C$ and since C is also in NP, C is NP-complete.

NP-completeness

- The most difficult part now is to find a problem that it is indeed NP-complete, in other words that it is in NP and any other problem in NP is reducible to it.
- This tough job was done by Stephen Cook. Cook's theorem says that SAT (satisfiability) is NP-complete.
- Now we can start reducing SAT to other NP problems and show that they are NP-complete and then we can use these problems to find more NP-complete problems.

NP-completeness

- Actually, if we knew that Independent Set is NP-complete we can show that Clique is also NP-complete:
 - Clique is in NP because we can verify in polynomial time a “yes” instance (a graph G that has a clique of size k). If we are given a set of nodes S of size k (or we guess it) we can check that it is indeed a clique by checking the adjacency matrix of G if any two elements i, j in S are connected ($A[i, j] = 1$). This needs time $O(k^2)$ which is $O(n^2)$ since k can be at most n .
 - We can show that Clique is NP-hard by using the aforementioned reduction from Independent Set to Clique